

第三题 航道清理

【问题描述】

在遥远的未来，人类开辟了一条重要的星际航道。航道是笔直的，起点为空间站“阿尔法”，终点为空间站“欧米伽”。

在这条航道中(起点和终点间)，散布着 N 块危险的小行星残骸。领航员在从阿尔法到欧米伽的航行过程中，必须在残骸处手动领航实施绕行（距离忽略不计），其它时间则交由自动领航程序航行。航行过程中多段连续自动领航航行距离的最小值，称为最小安全距离。为提升航行安全，航道管理部门决定清理一部分残骸，使得飞船在航行过程中的最小安全距离尽可能大。由于资源有限，最多只能清理 M 块残骸（起点和终点的空间站不能清理）。

【输入描述】

第一行包含三个整数 L, N, M ，分别表示航道的总长度（阿尔法到欧米伽的距离），航道中的残骸数量，以及最多可清理的残骸数量。保证 $L \geq 1$ 且 $N \geq M \geq 0$ 。接下来 N 行，每行一个整数，第 i 行的整数 $D_i (0 < D_i < L)$ 表示第 i 块残骸与起点阿尔法的距离。这些数据按照距离从小到大的顺序给出，且不会有两个残骸在同一位置。

【输出描述】

一个整数，表示能够实现的最小安全距离的最大值。

【输入样例 1】

```
25 5 2
2
11
14
17
21
```

【输出样例 1】

```
4
```

【数据范围】

对于 20% 的数据， $0 \leq M \leq N \leq 10$ 。

对于 50% 的数据， $0 \leq M \leq N \leq 100$ 。

对于 100% 的数据， $0 \leq M \leq N \leq 50000$ ， $1 \leq L \leq 10^9$ 。

【示例解释】

绕行过距离起点 2 和 14 的两块残骸后，飞船在航行过程中的最小安全距离为 4（例如从距离 17 的位置跳到 21，或从 21 跳到终点欧米伽）。

题目解析

题目分析：

本题题意为输入一个总长度 L ，元素个数 N ，以及可以去除的元素个数 M 。现在给定 N 个从小到大的数，现在要求去掉里面最多 M 个数，使得剩余的元素中相邻两个数的差的最小值尽量的大，求出这个值。 本题考察了二分答案以及贪心的知识。

解题思路：

对于 20% 的数据， $N \leq 10$ ，该数据范围下的 N 非常小，可以直接用 DFS 暴力枚举移除了那些残骸，然后计算距离的最小值，全局中统计 所有最小值的最大值。

进一步思考，如果最小的安全距离已经是一个确定的值，那么对于 N 个残骸，如果发现有两个相邻的残骸之间的距离小于的这个最小的安全距离，那么就一定要移除其中一个。这里可以有一个贪心思路，如果当前的和上一个的残骸的距离小于了最小的距离，一定是移除当前的是比较优的。因为之前的残骸之间已经满足了距离大于最小安全距离这个条件，所以移除前面的残骸是没有意义的，只有移除当前的残骸才有可能影响到和后面残骸的距离。

因此当最小的安全距离已经确定的情况下，可以通过上述贪心的策略计算出需要移除的残骸数量，如果超过了 M 那么这个距离就不可以。否则这个距离就可能是一个答案，结果需要求出这个最小安全距离的最大值。

可以考虑枚举这个距离，但是数据给的很大，可以发现当最小距离比较小的时候，一定可以满足移除 M 个以内的条件，最小距离比较大的情况下则一定不会满足，因此最小距离具备着单调性，可以使用二分答案的方式去枚举这个最小距离，时间复杂度为 $O(n \log L)$ 。可以通过 100% 的数据。

参考代码：

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

bool check(int minDist, const vector<int>& debris, int maxRemove, int totallLength)
{
    int lastPos = 0;
    int removeCount = 0;
    for (int i = 0; i < debris.size(); i++) {
        // 如果当前的元素和上一个留下来的元素的距离小于二分答案约定的最小距离的话就
        移除当前的元素。
    }
}
```

```

        if (debris[i] - lastPos < minDist) {
            removeCount++;
            //如果当前移除的元素个数超过了给定的移除数量则说明该距离不合适
            if (removeCount > maxRemove) {
                return false;
            }
        } else {
            // 不移除则更新上一个结束的位置
            lastPos = debris[i];
        }
    } // 最后还需要和终点的距离进行判断
    if (totalLength - lastPos < minDist) {
        removeCount++;
        if (removeCount > maxRemove) {
            return false;
        }
    }
    return true;
}

int main() {
    int L, N, M;
    cin >> L >> N >> M;

    vector<int> debris(N);
    for (int i = 0; i < N; i++) {
        cin >> debris[i];
    }

    int left = 1, right = L;
    int answer = 0;
    // 二分答案，查找最小安全距离的最大值
    while (left <= right) {
        int mid = left + (right - left) / 2;
        // 二分计算中点，让 mid 作为小安全距离的下限
        if (check(mid, debris, M, L)) {
            // 如果当前中点可以作为符合条件的小安全距离，那么可以尝试更大的答案，改变 left 的值
            answer = mid;
            left = mid + 1;
        } else {
            // 否则说明当前的最小安全距离选大了，需要尝试更小的答案，改变 right 的值
            right = mid - 1;
        }
    }
}

```

```
}  
  
cout << answer << endl;  
  
return 0;  
}
```